Projeto Final MAP 2026.1

Mini-Framework Orientado a Objetos para Jogos de Cartas

Padrões GoF Aplicados	Factory Method, Strategy (×3), Template Method e Observer — todos documentados e justificados no código.
Arquitetura	Framework 100% genérico (com.map.framework) separado dos jogos cliente (com.map.games), exposto via Facade.
SOLID + GRASP	SRP, OCP, LSP, ISP, DIP + Creator, Expert, Low Coupling, High Cohesion, Polimorfismo.
Jogos Implementados	Blackjack (21) como jogo principal e Super Trunfo de Carros Esportivos como alternativa.

Disciplina: Métodos Avançados de Programação

Java 17+ • Maven • JUnit 4 • Javadoc

Decisões Arquiteturais

Objetivo: criar um framework reutilizável de jogos de cartas (pacote `com.map.framework`), com dois jogos concretos como aplicações cliente (pacote `com.map.games`).

GameFacade centraliza a criação e execução de partidas:
`criarPartida(listaJogadores, regras)`
`executarPartida(partida)`
Oculta toda a complexidade interna do framework.

Orquestração do ciclo de vida da partida:
1. inicializar() → embaralha e zera estado
2. distribuir() → cartas iniciais (2 no Blackjack)
3. turnos() → loop de jogadas (HIT/STAND)
4. finalizar() → define o vencedor

Hooks: aposDistribuicao, antesDaRodada, aposRodada.

Arquitetura com generics ligando todos os pontos:
`Partida<B extends Baralho, R extends RegrasJogo<B>>`
`RegrasJogo<B>` cria o Baralho B do jogo.
Segurança de tipos em tempo de compilação.
Criar um novo jogo = implementar RegrasJogo<B> + estender Baralho/Carta.

1. Novos Baralhos via Factory Method
2. Novas RegrasJogo<B> (Strategy)
3. Novas EstrategiaJogador (IA, humano, remoto)
4. Novos EmbaralhadorStrategy
5. Novos PartidaListener (Observer)
6. Novas Cartas (CartaTradicional, CartaSuperTrunfo)
7. Novas Ações de jogador (AcaoJogador)

Padrões de Projeto GoF Aplicados

4 padrões GoF foram aplicados de forma explícita e integrada. Cada um tem um papel claro no código do framework.

Problema: criar cartas específicas de cada jogo.
Solução: Baralho.criarCartas() é abstrato; cada subclasse implementa:
`BaralhoTradicional` → 52 cartas (Blackjack)
`BaralhoSuperTrunfo` → 32 cartas de carros
Resultado: o framework não sabe quais cartas existem; a construção é delegada à subclasse.

1. EmbaralhadorStrategy:
EmbaralhadorAleatorio / SemEmbaralhamento
Permite trocar como o baralho é embaralhado em runtime.
2. EstrategiaJogador:
EstrategiaHumanaConsole (lê teclado)
EstrategiaIAAleatoria (decisão aleatória)
Jogador apenas delega para a estratégia (Context).
3. RegrasJogo<B>:
Regras plugáveis por jogo, usadas pela Partida.

Problema: todo jogo tem um fluxo semelhante; variantes no detalhe.
Solução: Partida.jogarPartida() é final (não pode ser sobrescrita)
e define o esqueleto fixo:
`inicializarPartida` → `distribuirCartas` → `rodarTurnos` → `finalizar`
A subclasse PartidaSimples e as regras (via hooks aposDistribuicao, antesDaRodada, aposRodada) variam os detalhes.

Problema: acoplamento entre a partida e o console (ou UI).
Solução: Partida é o Subject; gerencia a lista de `PartidaListener` (Observer).
Eventos: PARTIDA_INICIADA, RODADA_FINALIZADA, CARTA_DISTRIBUIDA, CARTA_JOGADA, JOGADOR_ELIMINADO, PARTIDA_FINALIZADA...
`PartidaConsoleListener` imprime os eventos no console.
Novas saídas (GUI, logs, JSON) = novos listeners, sem tocar na Partida.

Princípios SOLID e GRASP

Além dos padrões GoF, o código segue boas práticas de design, com justificativas documentadas nos Javadocs.

SRP — Jogador só representa estado; lógica de decisão fica na estratégia; o fluxo do jogo fica na Partida/Regras.
OCP — Carta/Baralho/Regras são fechados para modificação e abertos para novas especializações (novo jogo não altera o framework).
LSP — Subclasses de Baralho e Carta substituem a base sem alterar o comportamento esperado.
ISP — RegrasJogo<B> define só o que o jogo precisa; métodos de hook usam default methods (opcionais).
DIP — Partida depende de RegrasJogo (abstração); Jogador depende de EstrategiaJogador; nunca de classes concretas.

Creator — O Baralho cria as próprias cartas (Factory Method) e a Partida cria as notificações de eventos.
Information Expert — Baralho conhece suas cartas (comprar, embaralhar); Partida conhece o fluxo e jogadores; Regras conhecem as regras.
Low Coupling — Jogos dependem de interfaces, não de classes concretas; Facade reduz acoplamento com o cliente.
High Cohesion — Pacotes card/, player/, game/ com responsabilidades claras e coesas.
Polimorfismo — RegrasJogo e EstrategiaJogador variam o comportamento em tempo de execução.

- Ações com semântica clara: HIT, STAND.
- Nomes em português nas classes (Jogador, Mao, Baralho) e inglês/português nos métodos da API.
- Validação de pré-condições no construtor da Partida (ex: número de jogadores, regras não nulas).
- Documentação Javadoc completa (requisito do projeto).

- 20+ testes automatizados com JUnit 4: MaoTest, BaralhoTradicionalTest, BlackjackUtilsTest, JogadorEstrategiaTest, SuperTrunfoCartaTest.
- Build gerenciado pelo Maven (Java 17+, plugin Javadoc).
- Estrutura: pom.xml com foco no plugin surefire (testes).

Jogos Implementados

Dois jogos foram construídos sobre o **mesmo** framework, reutilizando 100% das classes genéricas.

Execução: jogo principal (executado via JAR).
Jogadores: humano (console) vs Dealer (IA aleatória).
Regras: 1 a 6 baralhos tradicionais; 2 cartas iniciais.
Ações: HIT (comprar) e **STAND** (parar).
Objetivo: chegar perto de 21 sem estourar.
Blackjack natural: 21 com as 2 primeiras cartas vence automaticamente (pontuação recebe +999).
Resultado: maior pontuação vence; estourou perde; empates.
Classe: com.map.games.blackjack.MainBlackjack

Execução: aplicação alternativa (rodar pela própria main).
Jogadores: humano (console) vs IA aleatória.
Regras: 32 cartas de carros esportivos; 5 atributos: potência, velocidade, aceleração, preço, cilindradas.
Mecânica: um jogador escolhe um atributo; quem tiver o melhor valor vence a rodada.
Carta Super Trunfo: ela vence qualquer outra (exceto valores mínimos do atributo escolhido).
Vitória: mais pontos ao fim das cartas ou após a 10ª rodada.
Classe: com.map.games.supertrunfo.MainSuperTrunfo

```
Execução: mvn clean package java -jar target/projeto-map-2026-1-1.0.0.jar # Blackjack java -cp target/...
com.map.games.supertrunfo.MainSuperTrunfo # Alternativa
```

Conclusão: o framework atende aos requisitos do projeto — 4 padrões GoF + SOLID/GRASP + 2 jogos cliente + testes e Javadoc.